# Advanced Function Concepts in JavaScript

# Function Parameters

- In JavaScript, **functions** are reusable blocks of code designed to perform a particular task.
- Functions often need input values to work with – these are called **parameters**.
- Parameters are a fundamental concept in programming and enable flexibility and reusability of code.

# What Are Function Parameters?

- Function parameters are **placeholders** used when defining a function.
- They allow you to **pass data into the function** when it's invoked (called).
- When you pass a value to a parameter during a function call, it becomes known as an **argument**.

```
function welcome(city) {
  console.log("Welcome to " + city + "!");
}

welcome("Kochi");
// Output: Welcome to Kochi!
```

# Types of Function Parameters

1. Required Parameters
2. Default Parameters
3. Rest Parameters(…)
4. Parameter Destructuring

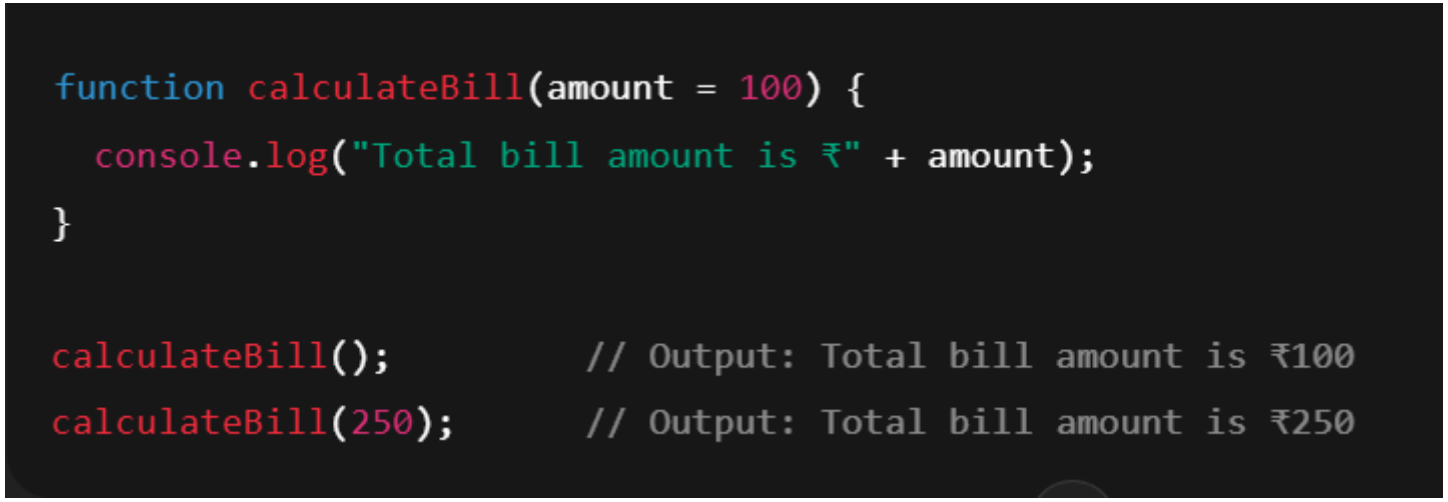
# 1. Required Parameters

- These are the most basic parameters. When the function is called, **arguments must be passed in the correct order** and quantity; otherwise, the result may be incorrect or undefined.
- **Key Points:**
- Parameters are required.
- If any argument is missing, the value becomes undefined.
- Parameters follow the order they're declared.

```
function add(a, b) {
  return a + b;
}
console.log(add(5, 3)); // Output: 8
```

# 2. Default Parameters (ES6)

- These parameters have predefined fallback values, used when no argument (or undefined) is passed for that parameter. Introduced in ES6.
- Key Points:
- Prevents undefined output.
- Enhances function flexibility.
- Default values can be any expression (string, number, function call, etc.).



```
function calculateBill(amount = 100) {
  console.log("Total bill amount is ₹" + amount);
}

calculateBill();          // Output: Total bill amount is ₹100
calculateBill(250);       // Output: Total bill amount is ₹250
```

# 3. Rest Parameters(…)

- Used when a function can accept **any number of arguments**. All extra arguments are collected into an array.
- **Key Points:**
- Collects excess arguments into an array.
- Useful when number of arguments is unknown.
- Must be placed **at the end** of the parameter list.

```
function findMax(...numbers) {
  return Math.max(...numbers);
}

console.log(findMax(10, 25, 3, 88, 45));
// Output: 88
```

# 4. Parameter Destructuring

- JavaScript allows object or array destructuring directly inside function parameters, making code more readable and clean.
- Types
    1. Object Destructuring in Function Parameters
    2. Array Destructuring in Function Parameters

# 1. Object Destructuring in Function Parameters

```
function displayStudent({ name, age, course }) {
  console.log(`Name: ${name}`);
  console.log(`Age: ${age}`);
  console.log(`Course: ${course}`);
}

const student = {
  name: "Vysakh",
  age: 24,
  course: "MERN Stack"
};

displayStudent(student);
```

# 2. Array Destructuring in Function Parameters

```
function showWeekdays([mon, tue, wed, thu, fri]) {
  console.log("Monday:", mon);
  console.log("Tuesday:", tue);
  console.log("Wednesday:", wed);
  console.log("Thursday:", thu);
  console.log("Friday:", fri);
}

const weekdays = ["Math", "English", "Science", "History", "Computer"];
showWeekdays(weekdays);
```

# Rest Parameters vs Spread Operator in Function Calls

| Feature | Rest Parameter | Spread Operator |
| --- | --- | --- |
| Purpose | Collect multiple args | Expand an array into individual args |
| Use in Function | **Definition** (inside () ) | **Call** (outside () ) |
| Output Type | Creates an **array** | Expands an **array** |
| Syntax Example | function fun(...args) | fun(...array) |

```
function markAttendance(...students) {
  console.log("Present Students:", students.join(", "));
  console.log("Total Present:", students.length);
}

const presentToday = ["Anjali", "Ravi", "Sneha", "Vysakh"];
markAttendance(...presentToday);
```

```
Present Students: Anjali, Ravi, Sneha, Vysakh
Total Present: 4
```

(...students) gathers all arguments into an array (rest parameter)
(....presentToday) spreads the array into individual values (spread operator).

# Function Hoisting

- Function hoisting means you can call a function before it is declared in your code. Only function declarations are hoisted, not function expressions.

```
sayWelcome();

function sayWelcome() {
  console.log("Welcome to Luminar Technolab!");
}
```

# Closures

- A closure is a function that remembers variables from its outer scope even after the outer function has finished executing.

```
function counter() {
  let count = 0;

  function increment() {
    count++;
    console.log("Count:", count);
  }

  return increment;
}

const increase = counter();
increase(); // Output: Count: 1
increase(); // Output: Count: 2
```

# Lexical Scope

- Lexical scope means a function has access to variables from its outer (parent) function based on where it is written in the code.

```
function outer() {
  let x = 10;

  function inner() {
    console.log(x); // x is accessible here
  }

  inner();
}

outer();
```

# Lexical Scope vs Closure

| Aspect | Lexical Scope | Closure |
| --- | --- | --- |
| **Definition** | Scope determined by where functions/variables are written | Function that remembers variables from its lexical scope |
| **When it occurs** | At the time of code writing | At the time of function execution and return |
| **Access** | Inner function can access variables from outer scope | Returned inner function retains access to outer variables |
| **Memory Usage** | No extra memory after execution | Variables are kept in memory even after outer function ends |
| **Use Case** | Managing visibility of variables | Creating private data, function factories, or callback handling |
| **Example** | Function inside another accessing outer variable | Function returned from another function with access to outer data |

# Pure Functions

- A pure function always gives the same output for the same input and does not modify any external state or variable.

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3)); // Output: 5
console.log(add(2, 3)); // Output: 5 (Same output every time)
```

- **Non-Pure Function Example**
- The function modifies the external variable total
- Same input gives different outputs because it depends on outside state

```
let total = 0;

function addToTotal(num) {
  total += num;
  return total;
}

console.log(addToTotal(5)); // Output: 5
console.log(addToTotal(5)); // Output: 10 (Different result for same input)
```

# Function Currying

- Currying is the process of converting a function with multiple arguments into a series of functions that each take one argument.

```
// Normal function
function add(a, b) {
  return a + b;
}

// Curried version
function curriedAdd(a) {
  return function(b) {
    return a + b;
  };
}

const addFive = curriedAdd(5);
console.log(addFive(3)); // Output: 8
```

# 6 Key Advantages of Currying

1. Code reusability – You can reuse curried functions with fixed arguments.
2. Partial application – Allows passing arguments step-by-step instead of all at once.
3. Improved readability – Makes code cleaner and easier to understand.
4. Modularity – Breaks complex functions into smaller, manageable parts.
5. Supports functional programming – Encourages a functional style of coding.
6. Works with closures – Helps understand and use closures effectively.

# Tasks

1. Create a function sayHi() with no parameters that logs “Hi there!”.
2. Write a function gradeScore(score) that returns:
3. "Excellent" if score ≥ 90
4. "Good" if score is between 75 and 89
5. "Needs Improvement" if score < 75
6. Create a function introduce(name = "Anonymous") that prints “My name is <name>”. Call it with and without an argument.
7. Make a function displayHobbies(...hobbies) that logs all hobbies passed as arguments.
8. Write a function showStudentDetails({ name, roll }) using object destructuring. Pass an object with those properties.
9. Create a function calculateTotal([price1, price2, price3]) using array destructuring. Return the sum of all three prices.
10. Build a function createAccount({ username, password = "1234" }). If password is not given, use default. Log both values.

# TASK

1. Create a program where a function is called before it's declared. Observe and explain if it works. (Function Hoisting)
2. Create a closure that stores a secret message and displays it only when a specific function is called.
3. Build a counter function using closures that increases the count each time it's called.
4. Write a nested function setup where the inner function accesses variables from its parent function. (Lexical Scope)
5. Create a pure function that takes two numbers and returns their sum without using or modifying any global variable.
6. Create a function that adds a tax rate to a product price using only pure function principles.
7. Convert a two-argument function (like addition or multiplication) into a curried version.
8. Create a curried function where the first call takes a user role (like "Admin", "Student") and the second call takes a name, then prints a custom welcome message.
9. Write a function that returns another function. The inner function should access and use a variable from the outer function. (Closure + Lexical Scope)
10. Create a function that gives a discount percentage first, and returns another function that accepts the price and prints the final price after discount. (Currying)